



FOUR-COURSE SYSTEM

COURSE 3 - DATA ENGINEERING 2026

C3-02 Transformation Engineering & Warehousing

Beginner-first canonical learner book | SQL + dbt Core + DuckDB | book sufficient without video

Artifact	C3_030_TRANSFORMATION_ENGINEERING_WAREHOUSING_BEGINNER_FIRST_LESSON_BOOK_RC1
Status	2026 - LEARNER EDITION
Phase	C3-02 Transformation Engineering & Warehousing 14 lessons 34-46 focused hours
Prerequisite	C3-01 Gate PASS. C2B warehouse/modeling foundations may be assumed; implementation depth is taught here.
Brand	Charcoal #1F2937 Teal #14B8A6 Soft Blue #3B82F6 AMOLED #000

House rule: analogy -> tiny example -> guided real tool -> break/fix -> independent changed version -> validation -> production reasoning -> evidence. A successful dbt command alone never proves correct data.

This book is your teacher

Use this book as the primary teacher. External videos and official documentation are visual or current-behavior companions, never hidden prerequisites.

The locked beginner-first lesson contract

- | | |
|-----------------------------------|---|
| 01. Why it matters | 09. Why it worked |
| 02. What you learn | 10. Common beginner mistakes |
| 03. Prerequisite / starting state | 11. If stuck / recovery |
| 04. Picture it in your head | 12. Small independent task |
| 05. New words before use | 13. Validate |
| 06. Useful video/source | 14. Teach it back |
| 07. Follow with me once | 15. Protected review + changed retry |
| 08. Expected result | 16. Evidence/mastery + exact next route |

Rules that protect your learning

- Save a genuine attempt before opening a protected review.
- After review, close it and solve a changed retry - do not copy the reviewed answer.
- A command that runs is not mastery: validate grain, keys, counts, state, failure/recovery and evidence.
- If a new engineering concept appears, this book explains it from first principles even when Bridge skills are assumed.
- When stuck, repair only the named weak competency; do not restart the whole phase.

Contents / Index

Actual page numbers after the master-audit repair. Mentor routes use these same pages.

Front matter	Page
This book is your teacher	2
Contents / Index	3
Module map + practice/review/Gate route	4
Phase-specific study notes	5
Lessons	Start
C3-02-L01 ETL vs ELT and transformation-layer boundaries	6
C3-02-L02 Production SQL style: readability and determinism	9
C3-02-L03 dbt project anatomy and dependency graph	12
C3-02-L04 Sources, staging models and naming contracts	15
C3-02-L05 Model materializations and rebuild strategy	18
C3-02-L06 Data tests: uniqueness, not-null, relationships and custom assertions	21
C3-02-L07 Incremental models and merge/upsert reasoning	24
C3-02-L08 Snapshots and SCD change-history patterns	27
C3-02-L09 Macros, templating and reusable transformation logic	30
C3-02-L10 Documentation, model descriptions and lineage artifacts	33
C3-02-L11 Dimensional model implementation and grain enforcement	36
C3-02-L12 Surrogate keys, late dimensions and referential integrity	39
C3-02-L13 Performance-aware SQL and query-plan reasoning	42
C3-02-L14 CI checks and curated serving-model handoff	45
Mastery checklist + revision quick reference	49

Module map + protected learning route

Transformation Engineering & Warehousing

- 01 Prerequisite / prior-phase Gate
- 02 Lesson mental model + vocabulary
- 03 Follow guided example once
- 04 Matching independent practice
- 05 Save genuine attempt
- 06 Protected review unlocks
- 07 Close review + changed retry
- 08 Validate + explain back
- 09 Master lesson + save evidence
- 10 Mini-Lab / integrated drill
- 11 Protected Gate A
- 12 If needed: protected review -> targeted repair -> changed micro-proof
- 13 Explicitly assign fresh Gate B/C only after repair evidence
- 14 Phase PASS -> next phase

What counts as mastery

- You can rebuild the competency without the lesson/review open.
- Your validation catches a plausible wrong result, not only syntax failure.
- You can state one failure mode and the recovery step.
- Your evidence names the input/output or artifact and the independent check used.

Revision rule

Revise from evidence: weak validation, weak recovery or weak explanation sends you back only to the matching lesson/repair route. Completion percentage never overrides a failed Gate.



How to study this phase

C3-02 assumes you can already reason about analytical SQL, dimensional models, ETL/ELT, SCD concepts and data quality from the Bridge. It does not assume you can implement a production-style transformation project. Each lesson therefore starts by saying what transfers and what changes in engineering.

Loop: Why -> Learn -> Prerequisite -> Picture -> New words -> Source/video -> Explanation -> tiny example -> Follow once -> expected result -> why -> break/fix -> changed independent task -> validation -> production reasoning -> teach-back -> protected review -> changed retry -> evidence.

Runtime: First dbt use is L03. Complete C3-029 first. Use the dedicated Python 3.13 `.venv-c3-02``; do not mutate another course venv.

C3-02-L01 - ETL vs ELT and transformation-layer boundaries

Focused effort: 2.0 h | Matching task: P01 | Review R01 stays protected until a genuine attempt is saved.

WHY IT MATTERS

Where transformation runs determines replay cost, trust boundaries, raw-data preservation and who can debug a failure. This is a deliberate Bridge revision because the concept is now used as an engineering design decision.

WHAT YOU LEARN

Defend ETL vs ELT for a concrete pipeline, state raw/staging/curated boundaries, and explain what remains immutable so a bad transformation can be replayed safely.

PREREQUISITE

C3-01 Gate PASS; C2B already introduced ETL/ELT as a warehouse concept.

PICTURE IT IN YOUR HEAD

Picture parcels entering a depot. You can open and repack them before storage (ETL), or store sealed originals first and repack copies inside the warehouse (ELT). Keeping the sealed original gives you a replay point.

NEW WORDS BEFORE WE USE THEM

Word	Meaning here
ETL	Extract, transform, then load to the target.
ELT	Extract, load raw/landing data, then transform in the target platform.
landing/raw	Closest preserved representation of what arrived.
curated	Validated data intentionally shaped for downstream use.
replay	Re-run processing from preserved inputs after a repair/change.

USEFUL VIDEO / SOURCE

RECOMMENDED REFERENCE: [IBM - ETL vs ELT](#)

Purpose: Current conceptual comparison; Bridge refresher, not a new prerequisite

Useful segment: Read comparison/when-to-use sections

OPTIONAL VIDEO: [IBM Technology - ETL vs ELT](#)

Purpose: Short visual refresher before transformation-layer design

Useful segment: Whole short concept video; Bridge concept only

PLAIN-ENGLISH EXPLANATION

ETL/ELT is not a fashion choice. Ask where compute lives, whether raw data must be retained, how sensitive data is governed, whether source changes must be replayed, and what failure boundary is easiest to operate. A common modern pattern lands immutable raw data, then transforms through staging/intermediate/mart layers.



C3-02-L01 - Independent transfer & mastery

INDEPENDENT SMALL VERSION

Design a changed support-ticket pipeline with raw JSON -> staging -> current ticket -> daily queue mart. Defend one ETL and one ELT choice.

VALIDATION - DO NOT SKIP

- Raw/source evidence is preserved.
- Each layer has an explicit grain.
- Business transformation location is justified.
- Replay starts from a named immutable/controlled point.

ARCHITECTURE / PRODUCTION REASONING

Later orchestration/lakehouse phases depend on this boundary. The same layer contract controls backfills, incident recovery, schema evolution and cost.

TEACH IT BACK

In 60-90 seconds explain etl vs elt and transformation-layer boundaries in simple English, including one failure mode and one independent validation.

PROTECTED REVIEW + CHANGED RETRY

1) Save P01 attempt in an append-only folder. 2) Only then open R01 in C3-032. 3) Record the specific misconception/failure. 4) Close review. 5) Complete the changed retry without copying. 6) If still weak, route only to targeted remediation.

EVIDENCE / MASTERY

Save boundary diagram, design decision, failure/replay note, changed ticket design and explain-back.

Mastered only when: changed retry passes, independent controls pass, you can explain without lesson/review open, and evidence is saved. A green dbt command by itself is not mastery.

C3-02-L02 - Production SQL style: readability and determinism

Focused effort: 2.5 h | Matching task: P02 | Review R02 stays protected until a genuine attempt is saved.

WHY IT MATTERS

Production SQL must return the same intended business state when ties, duplicate deliveries and larger datasets appear. "The query ran" is not a deterministic contract.

WHAT YOU LEARN

Write staged SQL with explicit projections/grain and deterministic latest-row logic; prove ties cannot select arbitrary winners.

PREREQUISITE

L01; advanced SQL/window foundations from C2B are assumed.

PICTURE IT IN YOUR HEAD

Imagine choosing the newest envelope from each customer. If two envelopes have the same timestamp and you have no second rule, the winner can be arbitrary. A source sequence is the tie-break stamp.

NEW WORDS BEFORE WE USE THEM

Word	Meaning here
deterministic	Same valid input/state produces the same intended result.
tie-breaker	Additional stable ordering field that resolves equal primary sort values.
projection	The exact columns selected into an output.
CTE	Named query stage used to make transformations inspectable.
grain contract	Statement of what one output row represents.

USEFUL VIDEO / SOURCE

RECOMMENDED REFERENCE: [DuckDB - SELECT](#)

Purpose: SQL semantics used in local transformation lab

Useful segment: Projection/filter/order behavior

RECOMMENDED REFERENCE: [PostgreSQL - Window Functions](#)

Purpose: Deterministic row-selection reasoning

Useful segment: Window ordering and row_number

PLAIN-ENGLISH EXPLANATION

Production SQL makes grain changes visible. Prefer explicit columns, clear aliases, small inspectable stages and deterministic ordering. `row_number()` ordered only by a non-unique timestamp is unsafe when timestamps tie.



C3-02-L02 - Follow with me once

TINY WORKED EXAMPLE

For two O002 rows with equal updated_at, `source\_seq=11` must beat `source\_seq=10`. Output grain becomes one current row per order_id.

FOLLOW WITH ME ONCE

1. Open `raw\_sources/orders\_base.csv`.
2. Find the intentionally tied O002 versions.
3. Build `ranked` CTE with row_number partitioned by order_id, ordered updated_at DESC, source_seq DESC.
4. Filter rn=1; assert one row/order_id.

TYPE / BUILD

```
with ranked as (  
  select order_id, status, updated_at, source_seq,  
         row_number() over (partition by order_id  
                           order by updated_at desc, source_seq desc) as rn  
  from raw_orders  
)  
select order_id, status, updated_at, source_seq from ranked where rn=1;
```

EXPECTED RESULT

Exactly one current winner per order_id; O002 winner is the row with the higher source_seq when updated_at ties.

WHY IT WORKED

Partition defines the business identity; ordered change time plus stable sequence defines which version wins; rn=1 materializes current truth.

BREAK / FIX / DEBUG

Break it by removing source_seq from ORDER BY. Add tied rows with different values. Fix by restoring a justified stable tie-breaker or changing the source contract.

COMMON BEGINNER MISTAKES

- SELECT * into durable marts.
- A timestamp-only latest-row rule when ties are possible.
- Using DISTINCT to hide duplicate business keys.
- No written input/output grain.

RECOVERY IF STUCK

Inspect duplicates first; print all candidate versions for one key; state what field proves recency; then rebuild ranking and uniqueness check.



C3-02-L02 - Independent transfer & mastery

INDEPENDENT SMALL VERSION

Use product-price changes instead of orders. Create a tied update and choose the current row deterministically.

VALIDATION - DO NOT SKIP

- One winner per business key.
- Tie fixture chooses expected winner.
- Repeated run on unchanged data returns identical keys/values.
- No DISTINCT used as fan-out repair.

ARCHITECTURE / PRODUCTION REASONING

Deterministic SQL is the foundation for incremental loads, SCD logic and replay. An unstable current-state model makes downstream incidents difficult to reproduce.

TEACH IT BACK

In 60-90 seconds explain production sql style: readability and determinism in simple English, including one failure mode and one independent validation.

PROTECTED REVIEW + CHANGED RETRY

1) Save P02 attempt in an append-only folder. 2) Only then open R02 in C3-032. 3) Record the specific misconception/failure. 4) Close review. 5) Complete the changed retry without copying. 6) If still weak, route only to targeted remediation.

EVIDENCE / MASTERY

Save SQL, tied fixture evidence, uniqueness control, changed product variant and 60-90 sec explanation.

Mastered only when: changed retry passes, independent controls pass, you can explain without lesson/review open, and evidence is saved. A green dbt command by itself is not mastery.



C3-02-L03 - dbt project anatomy and dependency graph

Focused effort: 2.5 h | Matching task: P03 | Review R03 stays protected until a genuine attempt is saved.

WHY IT MATTERS

dbt makes SQL transformations reviewable as a dependency graph instead of a folder of queries someone runs in the right order from memory.

WHAT YOU LEARN

Use a local dbt Core v1 project with DuckDB; explain project/profile/models/sources/tests/macros/snapshots and build dependencies with `source()`/`ref()`.

PREREQUISITE

L02. Git/venv/project structure from C3-00; no prior dbt use assumed.

PICTURE IT IN YOUR HEAD

Picture a subway map. Each model is a station; ``source()`` and ``ref()`` draw the tracks. dbt can then calculate which stations must build before others.

NEW WORDS BEFORE WE USE THEM

Word	Meaning here
dbt	Transformation framework that compiles project code into SQL and runs it against a data platform.
project	Version-controlled transformation code/config/tests/docs.
profile	Connection/runtime target configuration.
<code>ref()</code>	Dependency reference to another dbt model.
<code>source()</code>	Reference to a declared upstream source.
DAG	Directed acyclic graph of dependencies.

USEFUL VIDEO / SOURCE

RECOMMENDED REFERENCE: [dbt Developer Hub - Introduction](#)

Purpose: Current dbt product/core mental model and dependency graph vocabulary

Useful segment: Introduction + `ref`/`source` project concepts

RECOMMENDED VIDEO-PAGE: [dbt Labs - dbt explained \(official 2-minute explainer\)](#)

Purpose: See transformation-as-code, testing, docs and dependency workflow visually

Useful segment: Embedded 2-minute official explainer + How dbt works section

PLAIN-ENGLISH EXPLANATION

Current dbt docs expose both v2/Fusion and v1/Core tracks. This course deliberately uses dbt Core v1.11.x with dbt-duckdb 1.11.0 for a small reproducible local lab. The product generation is not the lesson; `source`/`ref`/`testing`/`materialization`/`lineage` concepts transfer.



C3-02-L03 - Follow with me once

TINY WORKED EXAMPLE

``stg_orders`` reads ``source(raw, orders)``. ``int_latest_orders`` uses ``ref(stg_orders)``. ``fct_order_lines`` depends on both current orders and staged order items.

FOLLOW WITH ME ONCE

1. Complete C3-029 first-use setup and activate ``.venv-c3-02``.
2. Run ``dbt --version`` and ``dbt debug --profiles-dir .``.
3. Open ``dbt_project.yml``, ``profiles.yml.example``, `models/`, `tests/`, `snapshots/`, `macros/`.
4. Run ``dbt parse`` then ``dbt ls --resource-type model``.
5. Draw the expected DAG before running models.

TYPE / BUILD

```
dbt --version
dbt debug --profiles-dir .
dbt parse
dbt ls --resource-type model
```

EXPECTED RESULT

``dbt debug`` reports a usable DuckDB connection; parse succeeds; model names list without running business transformations.

WHY IT WORKED

dbt reads project/profile metadata, parses Jinja/YAML/SQL, resolves dependency references and builds a DAG before execution.

BREAK / FIX / DEBUG

Break it: misspell a ``ref()`` target. ``dbt parse`/run` should fail before a trustworthy mart is published. Fix the dependency name; do not hardcode an accidental table path.

COMMON BEGINNER MISTAKES

- Confusing dbt with the database engine.
- Putting connection secrets in Git.
- Hardcoding upstream model names instead of `ref()`.
- Running everything before understanding the DAG.

RECOVERY IF STUCK

Run ``dbt parse`` and ``dbt ls`` first. Read the first meaningful error and identify whether it is project config, profile, Jinja/YAML or database connection.



C3-02-L03 - Independent transfer & mastery

INDEPENDENT SMALL VERSION

Add a tiny ``stg_regions`` source/model and a downstream ``dim_regions`` model using ``ref()``. Prove the dependency appears.

VALIDATION - DO NOT SKIP

- parse succeeds.
- debug succeeds when runtime available.
- Dependencies use `source()/ref()`.
- No secrets committed.

ARCHITECTURE / PRODUCTION REASONING

The DAG later powers selective builds, CI impact analysis, documentation/lineage, orchestration boundaries and incident triage.

TEACH IT BACK

Teach back: what does dbt control, what does DuckDB execute, and why does ``ref()`` matter beyond nicer syntax?

PROTECTED REVIEW + CHANGED RETRY

1) Save P03 attempt in an append-only folder. 2) Only then open R03 in C3-032. 3) Record the specific misconception/failure. 4) Close review. 5) Complete the changed retry without copying. 6) If still weak, route only to targeted remediation.

EVIDENCE / MASTERY

Save version/debug/parse output, DAG sketch, changed dependency proof and explain dbt vs DuckDB.

Mastered only when: changed retry passes, independent controls pass, you can explain without lesson/review open, and evidence is saved. A green dbt command by itself is not mastery.

C3-02-L04 - Sources, staging models and naming contracts

Focused effort: 2.5 h | Matching task: P04 | Review R04 stays protected until a genuine attempt is saved.

WHY IT MATTERS

If every model cleans raw data differently, downstream logic disagrees about what the source means. A thin staging contract creates one reviewable entry point.

WHAT YOU LEARN

Declare sources and build thin explicit staging models with names/types/required fields while avoiding premature business logic.

PREREQUISITE

L03; raw inputs from C3-01 are available.

PICTURE IT IN YOUR HEAD

Picture a customs desk at the warehouse entrance. It checks the package label, records what arrived and standardizes paperwork; it does not decide customer lifetime value.

NEW WORDS BEFORE WE USE THEM

Word	Meaning here
source declaration	Metadata describing an upstream relation dbt does not build.
staging model	Thin normalized entry model for one source entity.
naming contract	Stable rules for field/model names.
freshness	Whether source arrival is recent enough for intended use.
business logic	Rules deriving business state/metrics beyond source normalization.

USEFUL VIDEO / SOURCE

RECOMMENDED REFERENCE: [dbt - Sources](#)

Purpose: Declare raw inputs, `source()`, freshness/tests boundary

Useful segment: Source declarations + `source()`

RECOMMENDED REFERENCE: [dbt - Staging best practices](#)

Purpose: Current dbt staging structure and what belongs/does not belong in staging

Useful segment: Staging models + naming + no-joins/no-aggregations guidance

PLAIN-ENGLISH EXPLANATION

Staging should make source shape predictable: explicit columns, types, basic renames and low-risk normalization. Deduplication, current-state selection and cross-entity business rules usually belong downstream so raw meaning stays visible.



C3-02-L04 - Follow with me once

TINY WORKED EXAMPLE

`raw.orders` has `OrderID`, `UpdatedAt`, `SourceSeq`; `stg\_orders` outputs `order\_id`, `updated\_at`, `source\_seq` with explicit types but retains every delivered version.

FOLLOW WITH ME ONCE

1. Declare raw relations in `models/sources.yml`.
2. Replace `SELECT *` in `stg\_orders.sql` with explicit columns/casts.
3. State staging grain: one row per delivered order version.
4. Run/inspect source and staging counts; they should reconcile unless a documented filter exists.

TYPE / BUILD

```
select
  cast(order_id as varchar) as order_id,
  cast(updated_at as timestamp) as updated_at,
  cast(source_seq as bigint) as source_seq,
  cast(status as varchar) as status
from {{ source('raw', 'orders') }}
```

EXPECTED RESULT

Staging preserves the delivered order-version row count and required identity/change fields; column names/types are stable and explicit.

WHY IT WORKED

The source remains external truth; the staging node becomes the controlled transformation entry point and avoids repeated normalization downstream.

BREAK / FIX / DEBUG

Break it: deduplicate in staging with DISTINCT. A later history/incremental model loses source versions. Fix: restore delivered-version grain and move current-state logic to an intermediate model.

COMMON BEGINNER MISTAKES

- `SELECT *` in durable staging.
- Hiding invalid values with `COALESCE` defaults.
- Business aggregation in staging.
- Dropping source versions without a documented contract.

RECOVERY IF STUCK

Compare source and staging row counts/keys. If they differ, identify the exact clause that changes eligibility before adding any fix.



C3-02-L04 - Independent transfer & mastery

INDEPENDENT SMALL VERSION

Create staging for a changed shipments source with one invalid quantity. Preserve the invalid row visibly and add a validation flag rather than silently coercing to zero.

VALIDATION - DO NOT SKIP

- Explicit projection/types.
- Staging grain stated.
- Source/staging counts reconciled.
- Invalid/unmatched values remain visible.

ARCHITECTURE / PRODUCTION REASONING

Clean source boundaries make schema drift, lineage, tests and backfills diagnosable later. Thin staging is a governance decision as much as a style rule.

TEACH IT BACK

In 60-90 seconds explain sources, staging models and naming contracts in simple English, including one failure mode and one independent validation.

PROTECTED REVIEW + CHANGED RETRY

1) Save P04 attempt in an append-only folder. 2) Only then open R04 in C3-032. 3) Record the specific misconception/failure. 4) Close review. 5) Complete the changed retry without copying. 6) If still weak, route only to targeted remediation.

EVIDENCE / MASTERY

Save source YAML, staging SQL, count/grain controls, changed shipments variant and explanation.

Mastered only when: changed retry passes, independent controls pass, you can explain without lesson/review open, and evidence is saved. A green dbt command by itself is not mastery.

C3-02-L05 - Model materializations and rebuild strategy

Focused effort: 2.5 h | Matching task: P05 | Review R05 stays protected until a genuine attempt is saved.

WHY IT MATTERS

Materialization determines whether a model is recomputed, stored, incrementally updated or exposed as a view. The wrong choice can make a pipeline slow, stale or hard to recover.

WHAT YOU LEARN

Choose view/table/incremental/ephemeral-like strategies from data size, freshness, dependency, cost and rebuild/recovery requirements.

PREREQUISITE

L04.

PICTURE IT IN YOUR HEAD

Imagine recipe cards vs prepared meals. A view is a recipe evaluated when requested; a table is a prepared meal stored now; incremental adds only new/changed portions; a full rebuild cooks the whole batch again.

NEW WORDS BEFORE WE USE THEM

Word	Meaning here
materialization	How dbt persists or represents a model.
view	Stored query definition evaluated by the database.
table	Persisted result rebuilt when dbt runs it.
incremental	Persisted relation updated from a selected change set.
full refresh	Rebuild target from complete eligible source state.
rebuild strategy	Documented way to reconstruct trustworthy output.

USEFUL VIDEO / SOURCE

RECOMMENDED REFERENCE: [dbt - Materializations](#)

Purpose: View/table/incremental decision and rebuild implications

Useful segment: Materialization types and configuration

PLAIN-ENGLISH EXPLANATION

Choose the simplest materialization that meets operation needs. Views reduce storage but can repeat compute; tables stabilize expensive transforms but need refresh; incremental models reduce work but add state/replay complexity. Every incremental model needs a credible full-rebuild path.



C3-02-L05 - Follow with me once

TINY WORKED EXAMPLE

Use staging as views for tiny raw-normalization logic; use a mart table for a stable serving relation; use incremental only after you can identify changed business rows safely.

FOLLOW WITH ME ONCE

1. Inspect `dbt\_project.yml` materialization defaults.
2. For four example models, write size/freshness/rebuild reasoning before choosing materialization.
3. Change one model between view/table and inspect relation behavior.
4. Write a full-refresh/recovery rule for any incremental model.

TYPE / BUILD

```
# dbt_project.yml idea
models:
  c3_02_transform_lab:
    staging:
      +materialized: view
    marts:
      +materialized: table
```

EXPECTED RESULT

Each materialization choice has a reason tied to behavior/cost/recovery, not "dbt best practice says so."

WHY IT WORKED

Materialization controls when SQL executes and whether results persist; rebuild policy makes state recoverable after logic/schema mistakes.

BREAK / FIX / DEBUG

Break it: choose incremental for a tiny unstable model without a unique key/rebuild plan. Fix: revert to table until change identity and replay controls are proven.

COMMON BEGINNER MISTAKES

- Making everything incremental for speed.
- No full-refresh path.
- Treating view/table as semantic differences in business logic.
- Changing materialization without considering downstream consumers.

RECOVERY IF STUCK

Ask: can I rebuild from source? how much data? how often used? what is the freshness SLA? what state can become corrupt? Choose after answering.



C3-02-L05 - Independent transfer & mastery

INDEPENDENT SMALL VERSION

For changed finance models `stg\_payments`, `fct\_payments`, `monthly\_revenue`, choose materializations and justify a safe rebuild order.

VALIDATION - DO NOT SKIP

- Choice tied to explicit constraints.
- Incremental only with identity/change logic.
- Full rebuild path documented.
- Downstream impact considered.

ARCHITECTURE / PRODUCTION REASONING

At production scale, materialization interacts with compute cost, concurrency, warehouse sizing, backfill windows and incident recovery.

TEACH IT BACK

In 60-90 seconds explain model materializations and rebuild strategy in simple English, including one failure mode and one independent validation.

PROTECTED REVIEW + CHANGED RETRY

1) Save P05 attempt in an append-only folder. 2) Only then open R05 in C3-032. 3) Record the specific misconception/failure. 4) Close review. 5) Complete the changed retry without copying. 6) If still weak, route only to targeted remediation.

EVIDENCE / MASTERY

Save decision table, one materialization experiment, rebuild plan and changed finance reasoning.

Mastered only when: changed retry passes, independent controls pass, you can explain without lesson/review open, and evidence is saved. A green dbt command by itself is not mastery.

C3-02-L06 - Data tests: uniqueness, not-null, relationships and custom assertions

Focused effort: 3.0 h | Matching task: P06 | Review R06 stays protected until a genuine attempt is saved.

WHY IT MATTERS

Tests turn assumptions about grain and integrity into executable release controls. Without them, a green dbt run can publish wrong data.

WHAT YOU LEARN

Implement generic and custom data tests; intentionally make one test fail; repair the data/logic rather than weakening the test.

PREREQUISITE

L04-L05; C2B quality controls assumed conceptually.

PICTURE IT IN YOUR HEAD

Picture airport security gates. A successful flight schedule does not mean every passenger is valid. Each gate enforces a specific condition before data is allowed downstream.

NEW WORDS BEFORE WE USE THEM

Word	Meaning here
data test	Query/assertion that returns failure when data violates a rule.
uniqueness	One value/key occurrence at the intended grain.
not-null	Required field is present.
relationships	Foreign-key-like membership in an accepted parent set.
custom assertion	Business/integrity rule expressed as SQL/test logic.
false green	Pipeline says success because important correctness was never checked.

USEFUL VIDEO / SOURCE

RECOMMENDED REFERENCE: [dbt - Data tests](#)

Purpose: Built-in generic tests plus custom assertions

Useful segment: unique/not_null/relationships/accepted_values/custom

RECOMMENDED REFERENCE: [dbt - Unit tests](#)

Purpose: Understand data tests vs logic-focused unit tests

Useful segment: Concept boundary only

PLAIN-ENGLISH EXPLANATION

Tests must map to business/data contracts. Uniqueness tests the declared grain; not-null protects required identity; relationships expose orphan references; custom assertions protect rules such as nonnegative amounts. A test is not "annoying" when it catches a true defect.



C3-02-L06 - Follow with me once

TINY WORKED EXAMPLE

`fct\_order\_lines` must be unique on `(order\_id,line\_no)`, keys non-null, product/customer references controlled, quantity >0 and net amount >=0.

FOLLOW WITH ME ONCE

1. Add generic tests in schema YAML for required keys.
2. Create a custom SQL assertion for impossible quantity/amount.
3. Run tests and record the clean state.
4. Inject a duplicate or invalid quantity; prove the intended test fails.
5. Repair source/logic and rerun to green.

TYPE / BUILD

```
# concept - uniqueness at composite grain
select order_id,line_no,count(*) as n
from fct_order_lines
group by 1,2
having count(*) > 1;
```

EXPECTED RESULT

Clean fixture: zero failing rows. Injected duplicate: the intended grain test fails. After repair: test passes again without weakening/removing it.

WHY IT WORKED

The test asks a precise falsifiable question. Failure injection proves the check is capable of catching the class of defect it claims to guard.

BREAK / FIX / DEBUG

Break it: a relationship test fails for a late customer. Bad fix: disable test or inner-join away the fact. Correct fix may be a deliberate Unknown member/quarantine policy with monitored repair.

COMMON BEGINNER MISTAKES

- Testing surrogate key uniqueness but not business grain.
- Removing a failing test instead of diagnosing it.
- Using accepted_values for rules that need richer logic.
- No failure-injection evidence.

RECOVERY IF STUCK

Read the failing rows, not just the summary. State the intended grain/rule, isolate source vs transformation cause, repair, then rerun the same test.



C3-02-L06 - Independent transfer & mastery

INDEPENDENT SMALL VERSION

On a fresh returns model, create tests for return identity, required order reference and refund ≥ 0 ; inject one bad row and prove detection.

VALIDATION - DO NOT SKIP

- Every required rule has a matching test/control.
- At least one controlled failure is detected.
- Repair restores green without weakening checks.
- Tests align to declared grain.

ARCHITECTURE / PRODUCTION REASONING

CI later relies on these tests to block unsafe code/data changes. Tests are executable documentation and incident sensors.

TEACH IT BACK

In 60-90 seconds explain data tests: uniqueness, not-null, relationships and custom assertions in simple English, including one failure mode and one independent validation.

PROTECTED REVIEW + CHANGED RETRY

1) Save P06 attempt in an append-only folder. 2) Only then open R06 in C3-032. 3) Record the specific misconception/failure. 4) Close review. 5) Complete the changed retry without copying. 6) If still weak, route only to targeted remediation.

EVIDENCE / MASTERY

Save YAML/custom test, failing output, repair output, changed returns tests and explanation.

Mastered only when: changed retry passes, independent controls pass, you can explain without lesson/review open, and evidence is saved. A green dbt command by itself is not mastery.

C3-02-L07 - Incremental models and merge/upsert reasoning

Focused effort: 3.0 h | Matching task: P07 | Review R07 stays protected until a genuine attempt is saved.

WHY IT MATTERS

Incremental transformation saves work by processing change, but it creates state. If identity or change filters are wrong, reruns can duplicate rows or miss corrections.

WHAT YOU LEARN

Design a dbt incremental model with explicit unique identity, change selection, update/insert behavior, unchanged rerun proof and full-refresh recovery.

PREREQUISITE

L02, L05-L06; C3-01 incremental ingestion concepts are assumed.

PICTURE IT IN YOUR HEAD

Picture updating an address book. You do not rewrite every card each day, but you must know which person a changed card belongs to and whether to replace or insert it.

NEW WORDS BEFORE WE USE THEM

Word	Meaning here
incremental predicate	Rule selecting rows eligible for this run.
unique_key	Business/technical identity used to match existing target rows.
upsert	Update matched rows and insert new ones.
high-water mark	Saved boundary such as last processed update time.
late correction	Change arriving after an earlier processing boundary.
full refresh	Reconstruct target from complete source state.

USEFUL VIDEO / SOURCE

RECOMMENDED REFERENCE: [dbt - Incremental models](#)

Purpose: is_incremental, unique_key and safe change processing

Useful segment: Incremental model contract

RECOMMENDED VIDEO: [David Data - dbt incremental models](#)

Purpose: Visualize why incremental builds exist and how unique keys/change filters interact

Useful segment: 00:30-05:16 concepts; 09:21-11:17 implementation/full refresh

PLAIN-ENGLISH EXPLANATION

An incremental model needs two separate ideas: which rows changed, and which target identity they belong to. A timestamp filter without overlap/tie handling may miss late corrections. A merge without stable unique key may multiply rows. Always prove initial -> delta -> unchanged rerun -> full rebuild equivalence where feasible.

C3-02-L07 - Follow with me once

TINY WORKED EXAMPLE

Base contains O002. Delta changes O002 and adds O004. After applying delta once, O002 reflects update and O004 exists. Applying the same delta again changes no business identities/counts.

FOLLOW WITH ME ONCE

1. State target grain and unique_key.
2. Run baseline build and freeze keys/counts.
3. Apply `orders\_delta.csv`.
4. Run incremental build; inspect O002/O004.
5. Rerun same delta unchanged; compare keys/counts.
6. Document full-refresh path.

TYPE / BUILD

```
{{ config(materialized='incremental', unique_key='order_id') }}
select ...
from {{ ref('int_latest_orders') }}
{% if is_incremental() %}
where updated_at >= (select max(updated_at) from {{ this }})
{% endif %}
```

EXPECTED RESULT

One current row/order_id. O002 update applied exactly once; O004 inserted exactly once; unchanged rerun preserves identities and business totals.

WHY IT WORKED

Unique identity prevents duplicate target rows; change selection limits work; validation compares state transitions rather than trusting rowcount alone.

BREAK / FIX / DEBUG

Break it: use `>` max(updated_at) when a correction arrives exactly at the boundary. **Fix with a justified overlap/reprocessing window plus deterministic latest-row logic.**

COMMON BEGINNER MISTAKES

- Using ingestion timestamp as business identity.
- No unchanged-rerun test.
- No late-correction policy.
- Incremental path differs semantically from full refresh.

RECOVERY IF STUCK

Compare current target identities with the union of baseline+delta source identities; inspect one changed key end-to-end; if uncertain, full-refresh in a test target and compare.



C3-02-L07 - Independent transfer & mastery

INDEPENDENT SMALL VERSION

Build an incremental inventory-current model where an existing SKU quantity changes and a new SKU arrives. Prove same-delta rerun safety.

VALIDATION - DO NOT SKIP

- Unique target grain preserved.
- Required updates/inserts applied.
- Same delta does not multiply rows.
- Full rebuild/recovery documented and equivalent on fixture.

ARCHITECTURE / PRODUCTION REASONING

Incremental correctness is production state management. Later CDC/streaming/lakehouse MERGE patterns reuse the same identity, replay and reconciliation reasoning.

TEACH IT BACK

In 60-90 seconds explain incremental models and merge/upsert reasoning in simple English, including one failure mode and one independent validation.

PROTECTED REVIEW + CHANGED RETRY

1) Save P07 attempt in an append-only folder. 2) Only then open R07 in C3-032. 3) Record the specific misconception/failure. 4) Close review. 5) Complete the changed retry without copying. 6) If still weak, route only to targeted remediation.

EVIDENCE / MASTERY

Save baseline/delta/rerun evidence, target-key controls, recovery rule, changed inventory variant and explain-back.

Mastered only when: changed retry passes, independent controls pass, you can explain without lesson/review open, and evidence is saved. A green dbt command by itself is not mastery.

C3-02-L08 - Snapshots and SCD change-history patterns

Focused effort: 3.0 h | Matching task: P08 | Review R08 stays protected until a genuine attempt is saved.

WHY IT MATTERS

Current state answers "what is true now"; historical state answers "what was believed at a past time." Overwriting one with the other destroys auditability.

WHAT YOU LEARN

Use snapshot/SCD-style history to preserve descriptive changes; prove valid current and historical truth without inventing versions.

PREREQUISITE

L07; C2B SCD concept assumed but dbt implementation is new.

PICTURE IT IN YOUR HEAD

Picture a passport with old visas still visible. Your current address is one field; the history of prior addresses is a sequence of valid periods, not overwritten text.

NEW WORDS BEFORE WE USE THEM

Word	Meaning here
snapshot	dbt mechanism for recording changed rows over time.
SCD Type 2	History pattern storing multiple versions with validity ranges/current marker.
business key	Source identity such as customer_id.
valid_from	Time a historical version became effective/observed.
valid_to	Time it stopped being current; current row often remains open.
hard delete	Source row disappears rather than changing values.

USEFUL VIDEO / SOURCE

RECOMMENDED REFERENCE: [dbt - Snapshots](#)

Purpose: SCD Type-2 style history and snapshot strategies

Useful segment: Snapshot semantics + timestamp strategy

RECOMMENDED VIDEO: [Kahan Data Solutions - dbt snapshots](#)

Purpose: Visualize current row vs historical versions

Useful segment: 00:51-03:55 concepts; 03:55-09:29 build/change demo

PLAIN-ENGLISH EXPLANATION

History needs stable entity identity and a trustworthy change signal. The timestamp strategy is preferred when a reliable `updated_at` exists. Do not fabricate history before the first observation, and do not confuse load time with business effective time unless the contract says so.



C3-02-L08 - Follow with me once

TINY WORKED EXAMPLE

Customer BC01 starts tier=bronze. `customers\_v2.csv` changes BC01 to gold and adds BC04. Current dimension shows gold; snapshot history retains bronze and gold versions for BC01.

FOLLOW WITH ME ONCE

1. State entity business key and tracked attributes.
2. Create/run snapshot on v1.
3. Capture current/history controls.
4. Swap/apply v2 and run snapshot again.
5. Prove BC01 has two legitimate versions and BC04 appears once.

TYPE / BUILD

```
{% snapshot customers_snapshot %}
{{ config(unique_key='customer_id', strategy='timestamp', updated_at='updated_at') }}
select * from {{ source('raw', 'customers') }}
{% endsnapshot %}
```

EXPECTED RESULT

Changed customer has preserved old+new versions; unchanged customers do not gain false versions; current model reflects latest state.

WHY IT WORKED

Stable business key identifies entity; change metadata tells dbt when a new version is required; snapshot columns track validity.

BREAK / FIX / DEBUG

Break it: change a descriptive attribute but leave updated_at unchanged under timestamp strategy. Snapshot may not detect change. Fix the source contract or choose an appropriate strategy; do not invent timestamps silently.

COMMON BEGINNER MISTAKES

- Treating every load as a new historical version.
- Using mutable descriptive text as unique key.
- Overwriting history during current-state cleanup.
- Assuming source timestamp semantics without verification.

RECOVERY IF STUCK

For one entity, print every source/current/history version ordered by validity/change time. Explain why each version exists. If you cannot, stop and repair identity/change contract.



C3-02-L08 - Independent transfer & mastery

INDEPENDENT SMALL VERSION

Use subscription plan changes: S10 basic -> pro and new S12. Preserve history and prove current truth on a changed fixture.

VALIDATION - DO NOT SKIP

- Exactly expected versions per changed entity.
- Unchanged entities not duplicated.
- Current state agrees with latest legitimate source state.
- History dates/markers are interpretable.

ARCHITECTURE / PRODUCTION REASONING

History powers as-of analysis, audit, slowly changing dimensions and incident reconstruction. False history is as harmful as lost history.

TEACH IT BACK

In 60-90 seconds explain snapshots and scd change-history patterns in simple English, including one failure mode and one independent validation.

PROTECTED REVIEW + CHANGED RETRY

1) Save P08 attempt in an append-only folder. 2) Only then open R08 in C3-032. 3) Record the specific misconception/failure. 4) Close review. 5) Complete the changed retry without copying. 6) If still weak, route only to targeted remediation.

EVIDENCE / MASTERY

Save snapshot SQL/config, before/after version evidence, changed subscription example and explanation.

Mastered only when: changed retry passes, independent controls pass, you can explain without lesson/review open, and evidence is saved. A green dbt command by itself is not mastery.

C3-02-L09 - Macros, templating and reusable transformation logic

Focused effort: 2.5 h | Matching task: P09 | Review R09 stays protected until a genuine attempt is saved.

WHY IT MATTERS

Repeating the same SQL expression across dozens of models invites drift. Macros can centralize stable repeated structure, but too much abstraction makes compiled SQL hard to understand.

WHAT YOU LEARN

Use Jinja variables/macros for genuinely repeated transformation logic; inspect compiled SQL and know when not to abstract.

PREREQUISITE

L03-L08; basic Python-like logic helpful but Jinja is taught here.

PICTURE IT IN YOUR HEAD

Picture a rubber stamp. A macro prints the same tested shape wherever needed. But if every sentence becomes a stamp, nobody can read the document.

NEW WORDS BEFORE WE USE THEM

Word	Meaning here
Jinja	Template language dbt evaluates before SQL reaches the database.
macro	Reusable Jinja function that generates text/SQL.
compile	Render templates into database SQL.
DRY	Avoid unnecessary repetition; not a command to abstract everything.
compiled SQL	Actual SQL produced after Jinja/ref/source resolution.

USEFUL VIDEO / SOURCE

RECOMMENDED REFERENCE: [dbt - Jinja and macros](#)

Purpose: Reusable compile-time SQL generation

Useful segment: Jinja variables/macros/ref boundary

RECOMMENDED VIDEO: [Kahan Data Solutions - Jinja in dbt](#)

Purpose: See Jinja variables/conditions before writing a macro

Useful segment: 01:02-04:40 variables; 07:54-end conditions

PLAIN-ENGLISH EXPLANATION

dbt does not execute Jinja inside DuckDB. dbt renders templates first, then the database executes compiled SQL. A macro is valuable when logic is repeated and stable. Always inspect compiled SQL when behavior is surprising.



C3-02-L09 - Follow with me once

TINY WORKED EXAMPLE

Create ``net_amount(qty, price, discount_pct)`` macro used in two models. Compile and confirm the generated arithmetic is what you intended.

FOLLOW WITH ME ONCE

1. Open `macros/README_TASK.md`.
2. Implement one small macro from a clear contract.
3. Use it in a practice model.
4. Run ``dbt compile``.
5. Find rendered SQL under `target/compiled` and compare with handwritten expected expression.

TYPE / BUILD

```
{% macro net_amount(qty, price, discount) %}  
  ({{ qty }} * {{ price }} * (1 - {{ discount }}))  
{% endmacro %}
```

EXPECTED RESULT

Compiled SQL contains ordinary DuckDB SQL; the macro itself does not remain in the executed statement.

WHY IT WORKED

dbt/Jinja expands reusable template logic before dispatching SQL. Inspecting the compiled form separates template bugs from database SQL bugs.

BREAK / FIX / DEBUG

Break it: macro silently assumes discount is 0-100 while source stores 0-1. Fix the contract/test; do not hide unit assumptions inside abstraction.

COMMON BEGINNER MISTAKES

- Macro for a one-off expression.
- Macro name hides business units.
- Debugging only source template, never compiled SQL.
- Putting data validation inside opaque templating.

RECOVERY IF STUCK

Run compile, open smallest compiled model, replace macro mentally with generated SQL, and test expression on one known row.



C3-02-L09 - Independent transfer & mastery

INDEPENDENT SMALL VERSION

Create a changed ``safe_ratio(numerator,denominator)`` macro with explicit divide-by-zero behavior; compile and validate on three cases.

VALIDATION - DO NOT SKIP

- Compiled SQL matches intended expression.
- Known-row calculation matches manual control.
- Edge case behavior explicit.
- Macro reduces real repetition without hiding grain.

ARCHITECTURE / PRODUCTION REASONING

Macros become platform abstractions and policy helpers later. Over-abstraction can create organization-wide hidden coupling, so clarity beats cleverness.

TEACH IT BACK

In 60-90 seconds explain macros, templating and reusable transformation logic in simple English, including one failure mode and one independent validation.

PROTECTED REVIEW + CHANGED RETRY

1) Save P09 attempt in an append-only folder. 2) Only then open R09 in C3-032. 3) Record the specific misconception/failure. 4) Close review. 5) Complete the changed retry without copying. 6) If still weak, route only to targeted remediation.

EVIDENCE / MASTERY

Save macro, compile output, known-row check, changed `safe_ratio` example and teach-back.

Mastered only when: changed retry passes, independent controls pass, you can explain without lesson/review open, and evidence is saved. A green dbt command by itself is not mastery.

C3-02-L10 - Documentation, model descriptions and lineage artifacts

Focused effort: 2.5 h | Matching task: P10 | Review R10 stays protected until a genuine attempt is saved.

WHY IT MATTERS

A correct model nobody understands becomes tribal knowledge. Documentation must tell another engineer what a model means, its grain, owner/use and caveats; lineage explains dependencies but not business meaning.

WHAT YOU LEARN

Write model/column descriptions, generate/inspect dbt docs/lineage where runtime supports it, and create a handoff another engineer can operate.

PREREQUISITE

L03-L09.

PICTURE IT IN YOUR HEAD

Picture a city map plus street signs. The DAG is the map showing connections; descriptions are the signs explaining what each place is for. You need both.

NEW WORDS BEFORE WE USE THEM

Word	Meaning here
description	Human-readable model/column meaning stored near code.
lineage	Upstream/downstream dependency relationships.
catalog	Metadata about built relations/columns.
owner	Person/team accountable for data contract/operation.
caveat	Known limitation or usage boundary.
handoff	Evidence/instructions enabling another engineer to operate safely.

USEFUL VIDEO / SOURCE

RECOMMENDED REFERENCE: [dbt - Documentation](#)

Purpose: Descriptions/docs/catalog/lineage artifacts

Useful segment: Descriptions + docs generate/serve

RECOMMENDED VIDEO: [Mukesh Singh - dbt layered approach, docs and lineage](#)

Purpose: Current compact visual tying staging/intermediate/mart layers to documentation

Useful segment: 00:32-03:24 layers; 05:44-07:39 docs/lineage

PLAIN-ENGLISH EXPLANATION

Useful docs answer: purpose, one-row grain, key columns, source dependencies, freshness/refresh behavior, tests, owner, sensitive-data notes and known caveats. The lineage graph is derived from dependencies and helps impact analysis, but it cannot infer business semantics you never wrote.



C3-02-L10 - Follow with me once

TINY WORKED EXAMPLE

Document `fct\_order\_lines`: one row per paid current order_id+line_no, amount formula, Unknown customer handling, source dependencies, tests, known late-dimension caveat.

FOLLOW WITH ME ONCE

1. Add descriptions to schema YAML.
2. Run/inspect `dbt docs generate` if runtime available.
3. Open lineage and trace raw.orders -> staging -> current orders -> fact -> serving mart.
4. Write `HANDOFF.md` with setup/run/test/rebuild/failure sequence.

TYPE / BUILD

```
dbt docs generate
# optional local viewing when appropriate
dbt docs serve
```

EXPECTED RESULT

A reviewer can identify purpose/grain/keys/dependencies/tests/caveats without reading every SQL file; lineage matches ref/source dependencies.

WHY IT WORKED

Metadata sits beside version-controlled code; dbt can compile catalog/lineage artifacts from project + built state.

BREAK / FIX / DEBUG

Break it: description says one row/customer but a model actually has one row/order. Fix the model or documentation after verifying intended contract; never let docs become decorative.

COMMON BEGINNER MISTAKES

- Descriptions that repeat column names only.
- No grain statement.
- Calling DAG lineage a business definition.
- Handoff lacks clean-run/recovery order.

RECOVERY IF STUCK

Ask another-person questions: "What is one row? Can I rebuild it? What can fail? Who owns it?" If docs do not answer, improve docs before adding prose volume.



C3-02-L10 - Independent transfer & mastery

INDEPENDENT SMALL VERSION

Document a changed refunds mart and draw upstream/downstream lineage with one known caveat and one rebuild step.

VALIDATION - DO NOT SKIP

- Grain/purpose/key explicit.
- Lineage consistent with ref/source.
- Tests/rebuild/caveats discoverable.
- Another person can follow handoff.

ARCHITECTURE / PRODUCTION REASONING

Documentation and lineage later support CI impact analysis, governance, incident response, onboarding and system design interviews.

TEACH IT BACK

In 60-90 seconds explain documentation, model descriptions and lineage artifacts in simple English, including one failure mode and one independent validation.

PROTECTED REVIEW + CHANGED RETRY

1) Save P10 attempt in an append-only folder. 2) Only then open R10 in C3-032. 3) Record the specific misconception/failure. 4) Close review. 5) Complete the changed retry without copying. 6) If still weak, route only to targeted remediation.

EVIDENCE / MASTERY

Save YAML descriptions, lineage evidence or honest runtime boundary, HANDOFF, changed refunds docs and explanation.

Mastered only when: changed retry passes, independent controls pass, you can explain without lesson/review open, and evidence is saved. A green dbt command by itself is not mastery.

C3-02-L11 - Dimensional model implementation and grain enforcement

Focused effort: 3.5 h | Matching task: P11 | Review R11 stays protected until a genuine attempt is saved.

WHY IT MATTERS

C2B taught how to reason about facts/dimensions. Here you implement that contract so joins cannot silently multiply or drop facts.

WHAT YOU LEARN

Build dimensions and a fact at a declared grain, enforce uniqueness/relationships, reconcile facts before and after joins, and expose a serving mart without changing fact grain accidentally.

PREREQUISITE

C2B dimensional modeling mastery; L04-L10 implementation skills.

PICTURE IT IN YOUR HEAD

Picture Lego studs. A fact row has specific connector keys; each dimension key must connect to at most one intended member. If a dimension has duplicate keys, one fact can explode into multiple rows.

NEW WORDS BEFORE WE USE THEM

Word	Meaning here
fact table	Events/measurements at an explicit business grain.
dimension	Descriptive context with controlled identity.
surrogate key	Warehouse-managed key independent of mutable business descriptions.
conformed dimension	Reusable dimension with consistent meaning across facts.
fan-out	Join multiplies rows because supposed one-side is not unique.
reconciliation	Independent comparison proving identities/counts/measures are preserved as intended.

USEFUL VIDEO / SOURCE

RECOMMENDED REFERENCE: [Microsoft Fabric - Dimensional modeling overview](#)

Purpose: Current dimensional modeling reference for fact/dimension/grain implementation

Useful segment: Fact/dimension/grain design

OPTIONAL REFERENCE: [dbt - Introduction](#)

Purpose: Local project context; implementation remains course-built

Useful segment: Use only for dbt project context

PLAIN-ENGLISH EXPLANATION

Implementation starts by freezing grain before joins. A fact at order_id+line_no should stay at that grain after dimension lookups. Validate dimension uniqueness before joining; then compare fact row count, distinct grain keys and monetary totals before vs after.



C3-02-L11 - Follow with me once

TINY WORKED EXAMPLE

`fact\_order\_lines`: one row per current paid order line. Join `dim\_customers` and `dim\_products` only after their surrogate/business-key uniqueness is proven.

FOLLOW WITH ME ONCE

1. Write grain comments in every mart SQL.
2. Build dimensions; test surrogate and business identity policy.
3. Build fact from current eligible order lines before dimension joins.
4. Freeze row count/distinct grain keys/gross control.
5. Join dimensions and rerun same controls.

TYPE / BUILD

```
-- fact grain: one row per (order_id, line_no)
select o.order_id,i.line_no,c.customer_key,p.product_key,
       i.qty*i.unit_price as gross_amount
from {{ ref('int_latest_orders') }} o
join {{ ref('stg_order_items') }} i using(order_id)
left join {{ ref('dim_customers') }} c using(customer_id)
left join {{ ref('dim_products') }} p using(product_id)
```

EXPECTED RESULT

Fact row count equals distinct order_id+line_no count; joins do not change intended fact identities or reconciled gross amount.

WHY IT WORKED

Unique dimensions ensure each fact lookup yields at most one member; left/controlled-unknown handling prevents unintended fact loss.

BREAK / FIX / DEBUG

Break it: duplicate product business keys in the dimension. Fact rows fan out. Bad fix: DISTINCT on final fact. Fix dimension identity/history logic, then rebuild and reconcile.

COMMON BEGINNER MISTAKES

- Choosing fact grain after building joins.
- DISTINCT as a fan-out fix.
- Inner joining late/unmatched dimensions without policy.
- Testing dashboard totals but not fact identities.

RECOVERY IF STUCK

Back up to pre-join fact controls. Test each dimension key uniqueness independently. Add one dimension at a time and compare row/key/measure controls.



C3-02-L11 - Independent transfer & mastery

INDEPENDENT SMALL VERSION

Implement a changed shipment-item fact with carrier/service dimensions; inject a duplicate carrier key and demonstrate controlled failure.

VALIDATION - DO NOT SKIP

- Fact grain unique.
- Dimension identity valid.
- No unintended row/measure change after joins.
- Serving model derives from fact without corrupting grain.

ARCHITECTURE / PRODUCTION REASONING

Production warehouse models support many consumers. Grain errors propagate widely, so fact/dimension contracts and reconciliation are release blockers.

TEACH IT BACK

In 60-90 seconds explain dimensional model implementation and grain enforcement in simple English, including one failure mode and one independent validation.

PROTECTED REVIEW + CHANGED RETRY

1) Save P11 attempt in an append-only folder. 2) Only then open R11 in C3-032. 3) Record the specific misconception/failure. 4) Close review. 5) Complete the changed retry without copying. 6) If still weak, route only to targeted remediation.

EVIDENCE / MASTERY

Save model SQL/YAML, pre/post join controls, injected fan-out failure+repair, changed shipment variant and defense.

Mastered only when: changed retry passes, independent controls pass, you can explain without lesson/review open, and evidence is saved. A green dbt command by itself is not mastery.

C3-02-L12 - Surrogate keys, late dimensions and referential integrity

Focused effort: 3.0 h | Matching task: P12 | Review R12 stays protected until a genuine attempt is saved.

WHY IT MATTERS

Source business keys can be late, reused or changed. Facts must not disappear just because descriptive context has not arrived yet.

WHAT YOU LEARN

Implement deterministic surrogate identity and an explicit late-dimension/Unknown-member policy while preserving fact integrity and a repair path.

PREREQUISITE

L11; C2B surrogate/SCD concepts assumed.

PICTURE IT IN YOUR HEAD

Picture a parcel arriving before the customer profile card. You still record the parcel with a temporary "Unknown customer" shelf location, then relink it when the real profile arrives.

NEW WORDS BEFORE WE USE THEM

Word	Meaning here
business key	Identity provided by source system.
surrogate key	Warehouse-controlled dimension identity.
late-arriving dimension	Fact references a dimension member not yet available.
Unknown member	Deliberate dimension row used to preserve unresolved fact references.
referential integrity	Fact references resolve according to the model contract.
repair path	How unresolved references are updated once real dimension arrives.

USEFUL VIDEO / SOURCE

RECOMMENDED REFERENCE: [Microsoft Fabric - Load tables in a dimensional model](#)

Purpose: Surrogate keys, SCD/load behavior and late-reference reasoning

Useful segment: Surrogate/SCD loading concepts

PLAIN-ENGLISH EXPLANATION

Never silently drop late facts with an inner join. A common warehouse policy maps unresolved business keys to a monitored Unknown surrogate key. The unresolved business key must remain recoverable so later dimension arrival can repair the association.

C3-02-L12 - Follow with me once

TINY WORKED EXAMPLE

Fixture `late\_order.csv` references customer C999 before C999 exists. Fact remains present with Unknown customer_key; unresolved key count is monitored. When C999 arrives, rebuild/reprocess maps it correctly.

FOLLOW WITH ME ONCE

1. Create a deterministic Unknown dimension member.
2. Load/rebuild fact with C999 fixture.
3. Prove fact count/amount unchanged and unresolved count=1.
4. Add real C999 dimension row.
5. Run documented repair/rebuild and prove unresolved count returns to 0 without fact duplication.

TYPE / BUILD

```
coalesce(c.customer_key, -1) as customer_key
-- preserve source business key separately for repair/diagnosis
```

EXPECTED RESULT

Late fact remains in fact table, maps to Unknown initially, and later resolves to the real member without changing fact identity/measure.

WHY IT WORKED

Left lookup plus controlled fallback preserves the event; retained business key enables later repair; reconciliation prevents hidden loss.

BREAK / FIX / DEBUG

Break it: use inner join to dimension. C999 fact vanishes and totals decrease. Fix join/policy and add an unresolved-key monitor/test.

COMMON BEGINNER MISTAKES

- Hashing mutable description into surrogate identity.
- Unknown key without preserving unresolved source key.
- Permanent Unknown mapping with no repair process.
- Treating unresolved dimensions as zero-value facts.

RECOVERY IF STUCK

Compare source fact identities vs target; list unresolved business keys explicitly; do not "fix" by deleting rows. Trace one late key from source to Unknown to repaired state.



C3-02-L12 - Independent transfer & mastery

INDEPENDENT SMALL VERSION

Use a changed claim fact referencing a late provider P77. Preserve claim, monitor unresolved provider, then repair when provider arrives.

VALIDATION - DO NOT SKIP

- No fact identity/measure loss.
- Unknown mapping explicit and monitored.
- Real dimension arrival repairs mapping.
- No duplicate fact/history fabricated.

ARCHITECTURE / PRODUCTION REASONING

Late dimensions are normal in distributed systems. Explicit unknown/repair policy prevents ingestion timing from silently changing business totals.

TEACH IT BACK

In 60-90 seconds explain surrogate keys, late dimensions and referential integrity in simple English, including one failure mode and one independent validation.

PROTECTED REVIEW + CHANGED RETRY

1) Save P12 attempt in an append-only folder. 2) Only then open R12 in C3-032. 3) Record the specific misconception/failure. 4) Close review. 5) Complete the changed retry without copying. 6) If still weak, route only to targeted remediation.

EVIDENCE / MASTERY

Save C999 before/after evidence, unresolved control, repair path, changed claim example and explanation.

Mastered only when: changed retry passes, independent controls pass, you can explain without lesson/review open, and evidence is saved. A green dbt command by itself is not mastery.

C3-02-L13 - Performance-aware SQL and query-plan reasoning

Focused effort: 3.0 h | Matching task: P13 | Review R13 stays protected until a genuine attempt is saved.

WHY IT MATTERS

A transformation can be correct on 20 rows and unaffordable on 2 billion. Engineers must notice scan/join/sort/materialization behavior without "optimizing" away correctness.

WHAT YOU LEARN

Run EXPLAIN on a non-trivial query, identify major operators/cardinality movement, make one safe change, and prove output equivalence.

PREREQUISITE

L02 and L11-L12. Index internals are not required; plan vocabulary taught here.

PICTURE IT IN YOUR HEAD

Picture an airport route map. A query plan shows which checkpoints data passes through and roughly how much traffic flows between them. You look for huge detours, not just the final arrival time.

NEW WORDS BEFORE WE USE THEM

Word	Meaning here
query plan	Database execution strategy for SQL.
scan	Read rows/columns from a relation.
join operator	Plan step combining relations.
cardinality	Number of rows flowing through a step.
predicate pushdown	Apply filters earlier when the engine can safely do so.
equivalence	Optimization preserves intended output semantics.

USEFUL VIDEO / SOURCE

RECOMMENDED REFERENCE: [DuckDB - EXPLAIN](#)

Purpose: Local query-plan authority

Useful segment: EXPLAIN operators/cardinality/scan

RECOMMENDED REFERENCE: [PostgreSQL - Using EXPLAIN](#)

Purpose: Transferable plan-reading concepts

Useful segment: Plan nodes/costs/row estimates

OPTIONAL VIDEO: [Alex The Analyst - SQL Beginner to Advanced](#)

Purpose: Preserved broad SQL visual; not authority for DuckDB query plans

Useful segment: Use only relevant advanced-SQL/project section, not plan semantics

PLAIN-ENGLISH EXPLANATION

EXPLAIN is evidence about how the database plans a query, not a universal "good/bad" score. Start with grain/correctness. Then inspect large scans, join cardinality, sorts and repeated compute. A performance change is valid only if outputs remain equivalent on required cases.



C3-02-L13 - Follow with me once

TINY WORKED EXAMPLE

Explain a serving query that joins fact to dimensions and filters paid orders. Record scan/join operators and estimated/actual cardinality where available.

FOLLOW WITH ME ONCE

1. Freeze original query output keys/count/measure.
2. Run `EXPLAIN` (or EXPLAIN ANALYZE only when safe for the statement).
3. Identify one operator/cardinality observation.
4. Make one safe change such as earlier projection/filter or materialization choice.
5. Re-run output controls and compare.

TYPE / BUILD

```
EXPLAIN
select c.region, sum(f.gross_amount)
from fct_order_lines f
join dim_customers c using(customer_key)
where f.status='paid'
group by c.region;
```

EXPECTED RESULT

You can name at least one scan/join/aggregate step and prove the changed query preserves required output identities/aggregates.

WHY IT WORKED

The plan reveals physical strategy/cardinality flow; independent result controls protect semantics while you experiment.

BREAK / FIX / DEBUG

Break it: add an early filter that accidentally excludes late/Unknown records and makes query faster. **Fix by restoring semantic eligibility; speed never overrides correctness.**

COMMON BEGINNER MISTAKES

- Reading cost numbers without understanding operator.
- Optimizing tiny fixtures by folklore.
- Changing joins and assuming totals unchanged.
- Using EXPLAIN ANALYZE on unsafe modifying SQL.

RECOVERY IF STUCK

First validate output. Then simplify query to smallest plan. Compare operators before/after one change. If output differs, revert and diagnose semantics.



C3-02-L13 - Independent transfer & mastery

INDEPENDENT SMALL VERSION

On a changed returns-by-product query, run EXPLAIN, make one safe readability/performance change and prove row/amount equivalence.

VALIDATION - DO NOT SKIP

- Plan evidence actually executed or marked pending honestly.
- At least one operator/cardinality observation.
- Output controls identical after safe change.
- No correctness sacrificed for speed.

ARCHITECTURE / PRODUCTION REASONING

At scale, query plans influence warehouse cost, SLA, partitioning/materialization choices and incident diagnosis. This phase teaches reasoning, not engine-specific tuning tricks.

TEACH IT BACK

In 60-90 seconds explain performance-aware sql and query-plan reasoning in simple English, including one failure mode and one independent validation.

PROTECTED REVIEW + CHANGED RETRY

1) Save P13 attempt in an append-only folder. 2) Only then open R13 in C3-032. 3) Record the specific misconception/failure. 4) Close review. 5) Complete the changed retry without copying. 6) If still weak, route only to targeted remediation.

EVIDENCE / MASTERY

Save before/after SQL, EXPLAIN evidence, equivalence controls, changed returns example and explanation.

Mastered only when: changed retry passes, independent controls pass, you can explain without lesson/review open, and evidence is saved. A green dbt command by itself is not mastery.

C3-02-L14 - CI checks and curated serving-model handoff

Focused effort: 3.0 h | Matching task: P14 | Review R14 stays protected until a genuine attempt is saved.

WHY IT MATTERS

If tests/docs only run on one developer laptop, unsafe changes can reach shared data. CI makes change checks repeatable; a serving-model handoff tells consumers exactly what they can trust.

WHAT YOU LEARN

Design a CI gate for parse/build/test/static checks, separate safe synthetic CI from protected production secrets, and hand off curated models with explicit contracts.

PREREQUISITE

L03-L13; Git branch/PR workflow from C3-00.

PICTURE IT IN YOUR HEAD

Picture a factory exit gate. Every changed component passes the same checklist before leaving. The shipping label then tells the recipient what the finished item is, how fresh it is and what not to assume.

NEW WORDS BEFORE WE USE THEM

Word	Meaning here
CI	Automated checks triggered by code changes before integration/deployment.
workflow	Version-controlled CI job definition.
artifact	Saved output/evidence from a job.
secret	Credential/value stored outside repository source.
serving model	Curated relation intentionally exposed to consumers.
change safety	Evidence a code/config change did not break required contracts.

USEFUL VIDEO / SOURCE

RECOMMENDED REFERENCE: [dbt - Continuous integration](#)

Purpose: CI/change-safety concepts for dbt projects

Useful segment: CI checks + safe changes

RECOMMENDED REFERENCE: [GitHub - Build and test Python](#)

Purpose: Workflow syntax/CI testing reference

Useful segment: Workflow + pytest job pattern

PLAIN-ENGLISH EXPLANATION

CI should fail loudly on parse/test/static contract failures and must not expose production data or credentials. Local DuckDB synthetic fixtures make a good teaching CI target. A serving handoff includes grain, keys, freshness/build expectations, tests, owner/caveats and rollback/rebuild route.



C3-02-L14 - Follow with me once

TINY WORKED EXAMPLE

Pull request changes `fct\_order\_lines`. CI creates a clean venv, installs pinned requirements, runs static fixture audit, dbt parse/build/test on synthetic DuckDB, and stores logs. Handoff states fact grain and rebuild command.

FOLLOW WITH ME ONCE

1. Inspect `.github/workflows/ci.yml.example`.
2. List exact checks and what defect each catches.
3. Intentionally make a static fixture test fail locally, then repair.
4. Write a CI-safe secret/data policy.
5. Complete HANDOFF.md for one serving model.

TYPE / BUILD

```
# conceptual CI sequence
python -m pip install -r requirements-c3-02.txt
python scripts/static_fixture_audit.py
dbt debug --profiles-dir .
dbt build --profiles-dir .
# fail job if any required step fails
```

EXPECTED RESULT

Every required check has a fail condition; no production secret/data is needed; handoff gives another engineer enough information to operate/rebuild.

WHY IT WORKED

CI recreates the environment and executes the same contract from clean state; serving docs communicate the downstream boundary after checks pass.

BREAK / FIX / DEBUG

Break it: CI logs a database password or uses real customer export. Fix by synthetic fixtures and secret store/environment boundaries; rotate any exposed real secret.

COMMON BEGINNER MISTAKES

- CI that always exits green after ignored failures.
- Using production data for a basic PR test.
- No dependency pin/recreation.
- Serving model shared with no grain/freshness caveat.

RECOVERY IF STUCK

Run the CI commands manually from a clean local folder. If they require undocumented state, your CI contract is incomplete. Inspect exit code for every required step.



C3-02-L14 - Independent transfer & mastery

INDEPENDENT SMALL VERSION

Create a changed PR-check plan for `mart\_daily\_returns` and write its consumer handoff, including one rollback/rebuild instruction.

VALIDATION - DO NOT SKIP

- Required checks are blocking.
- Clean environment is reproducible.
- No real secrets/private data in repo/fixtures.
- Serving contract states grain/keys/freshness/tests/caveats/rebuild.

ARCHITECTURE / PRODUCTION REASONING

Later DataOps/cloud phases expand CI into deployment, environments, observability and access controls. The core rule remains: repeatable evidence gates changes.

TEACH IT BACK

In 60-90 seconds explain ci checks and curated serving-model handoff in simple English, including one failure mode and one independent validation.

PROTECTED REVIEW + CHANGED RETRY

1) Save P14 attempt in an append-only folder. 2) Only then open R14 in C3-032. 3) Record the specific misconception/failure. 4) Close review. 5) Complete the changed retry without copying. 6) If still weak, route only to targeted remediation.

EVIDENCE / MASTERY

Save CI workflow/plan, failure+repair evidence, secret/data policy, serving handoff, changed variant and interview defense.

Mastered only when: changed retry passes, independent controls pass, you can explain without lesson/review open, and evidence is saved. A green dbt command by itself is not mastery.



C3-02 exit checkpoint

After all 14 lessons are mastered, complete C3-033 BrightBasket. Gate A remains locked until every required Mini-Lab competency passes. Protected B/C are fresh full retests only by explicit assignment.

C3-03 is not unlocked by an average score or completion percentage. C3-034 Gate PASS requires every required transformation competency PASS and zero critical failures.

Mastery checklist + revision quick reference

- ☐ Every lesson: guided rebuild completed, genuine independent attempt saved, validation recorded.
- ☐ Protected review opened only after the attempt; changed retry solved after closing the review.
- ☐ Explain-back: business/engineering purpose, grain or state, one failure mode, one recovery step, one validation check.
- ☐ Mini-Lab / integrated drill completed with reproducible evidence.
- ☐ Gate A attempted independently. If NEEDS_REPAIR: review viewed, targeted repair completed, changed micro-proof recorded, then a fresh B/C assigned explicitly.
- ☐ No unresolved critical failure, secret/private data leakage, fabricated live-run evidence or copied Gate solution.
- ☐ Evidence is saved with enough context for another person to reproduce or inspect it.

Revision quick reference

Wrong result / wrong grain	Return to validation + data contract / grain lesson; reproduce smallest failing case.
Works once, rerun breaks	Return to idempotence/state/checkpoint/recovery lesson; prove clean replay.
Cannot explain failure	Return to mental model + break/fix section; write the failure boundary in your own words.
Gate NEEDS_REPAIR	Open only its protected review, repair named weak competency, solve changed micro-proof, then assign fresh retest.
Tool/vendor changed	Use the current official source for syntax/UI, but keep the book's durable engineering contract and validation.